

Synthèse – Deep Learning

MLP, CNN, RNN / LSTM / GRU et Seq2Seq

Fiche pédagogique structurée en français

Nom et prénom : Salah Eddine Elmechrafi
Module : Deep Learning
Institution : EMSI
Date : Juin 2026

Objectif de ce document

Cette synthèse fusionne en un seul document les notions essentielles des trois notebooks académiques :

1. **DL_MLP** – Perceptron multicouche, initialisation, boucle d'entraînement PyTorch, évaluation sur données tabulaires.
2. **CNN** – Convolution 2D, pooling, architecture LeNet-5, classification d'images (CIFAR-10), comparaison MLP/CNN.
3. **RNN** – Modèles de langage, RNN/LSTM/GRU, BPTT, architecture Seq2Seq, décodage glouton et beam search.

L'objectif est d'offrir une progression claire : apprendre, voir, puis séquencer.

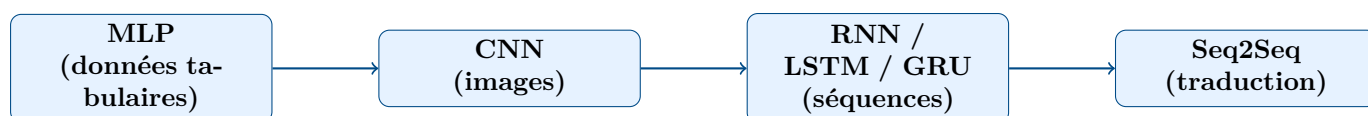
Table des matières

1	Vue d'ensemble	3
2	Partie I – Perceptron Multicouche (MLP)	3
2.1	Objectif et jeu de données	3
2.2	Architecture	3
2.3	Initialisation des poids	3
2.4	Boucle d'entraînement PyTorch	3
2.5	Sauvegarde et rechargement du modèle	4
2.6	Évaluation et résultats	4
3	Partie II – Réseau de Neurones Convolutif (CNN)	4
3.1	Idée centrale	4
3.2	Corrélation croisée 2D	4
3.3	Max-Pooling	5
3.4	Architecture LeNet-5 (CIFAR-10)	5
3.5	Préparation des données CIFAR-10	6
3.6	Visualisation des feature maps	6
3.7	Comparaison CNN vs MLP	6
4	Partie III – Modèles Séquentiels (RNN / LSTM / GRU)	6
4.1	Modèles de langage	6
4.1.1	Objectif probabiliste	6
4.1.2	Perplexité	7
4.2	Réseau neuronal récurrent (RNN)	7
4.2.1	Équation de récurrence	7
4.3	Apprentissage : BPTT et gradient clipping	7
4.3.1	Rétropropagation à travers le temps (BPTT)	7
4.3.2	Gradient clipping	8
4.4	LSTM : Long Short-Term Memory	8
4.4.1	Équations des portes	8
4.5	GRU : Gated Recurrent Unit	9
4.5.1	Comparaison LSTM / GRU	9
4.6	Variantes architecturales	9
4.6.1	RNN profonds	9
4.6.2	RNN bidirectionnels	9
4.7	Analyse comparative des modèles de langage	9
5	Partie IV – Séquence-à-séquence (Seq2Seq) et Traduction	10
5.1	Préparation des données	10
5.1.1	Pipeline	10
5.1.2	Tokens spéciaux	10
5.1.3	Perte masquée	10
5.2	Architecture encodeur-décodeur	10
5.3	Seq2Seq récurrent	10
5.3.1	Encodeur LSTM bidirectionnel	10
5.3.2	Teacher Forcing	11
5.3.3	Inférence auto-régressive	11
5.4	Stratégies de décodage	11
5.4.1	Décodage glouton (<i>Greedy</i>)	11
5.4.2	Score d'une séquence	11

5.4.3	Beam Search	12
5.4.4	Pénalisation de longueur	12
5.5	Évaluation : score BLEU	12
6	Comparaison globale des architectures	13
7	Mini-résumé opérationnel	13
8	Squelette d'entraînement Seq2Seq complet	13
9	Conclusion	14

1. Vue d'ensemble

Les trois notebooks s'inscrivent dans une progression pédagogique cohérente.



Chaque étape introduit une inductive bias adaptée à la structure des données : partage de poids dans l'espace pour le CNN, partage de poids dans le temps pour le RNN, puis séparation encodeur-décodeur pour la traduction.

2. Partie I – Perceptron Multicouche (MLP)

2.1. Objectif et jeu de données

Le notebook `DL_MLP.ipynb` vise à classer des tumeurs (bénignes ou malignes) à partir du dataset *Breast Cancer Wisconsin* de scikit-learn. Ce jeu de données contient 569 échantillons décrits par 30 caractéristiques numériques (rayon, texture, périmètre, etc.).

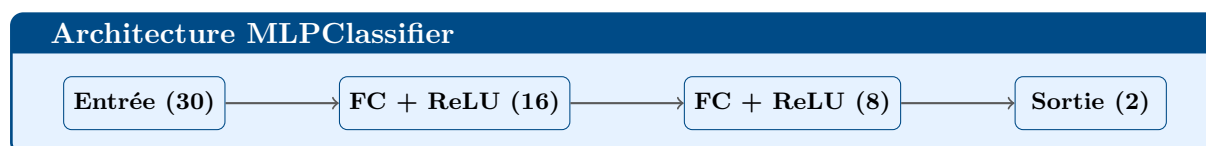
Le découpage des données est le suivant :

Ensemble	Proportion	Rôle
Train	70%	Mise à jour des poids
Validation	15%	Réglage des hyperparamètres
Test	15%	Évaluation finale

Attention

La normalisation (`StandardScaler`) est indispensable avant d'alimenter un MLP : sans elle, les gradients des features à grande variance dominent ceux des features à petite variance, ce qui empêche la convergence.

2.2. Architecture



2.3. Initialisation des poids

Stratégie	Formule	Usage recommandé
Gaussienne	$\mathcal{N}(0, 0.01^2)$	Expérimental / baseline
Constante	$w = 0.1$	À éviter (bris de symétrie impossible)
Xavier / Glorot	$\mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}\right]$	ReLU / Tanh

L'initialisation **Xavier** est appliquée dans le notebook avec `nn.init.xavier_uniform_`.

2.4. Boucle d'entraînement PyTorch

La boucle d'entraînement respecte les **cinq étapes canoniques** :

```

for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()           # 1. Remise à zéro des gradients
    outputs = model(X_train_t)      # 2. Forward pass
    loss = criterion(outputs, y_t)  # 3. Calcul de la perte
    loss.backward()                 # 4. Rétropropagation
    optimizer.step()                 # 5. Mise à jour des poids

    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val_t), y_val_t)

```

Listing 1 – Boucle d'entraînement standard (150 époques)

Optimiseur : Adam ($\alpha = 0.01$). **Perte :** CrossEntropyLoss.

2.5. Sauvegarde et rechargement du modèle

```

torch.save(model.state_dict(), "model_initial.pth")
loaded_model = MLPClassifier(input_dim, hidden_dim, output_dim)
loaded_model.load_state_dict(torch.load("model_initial.pth"))
loaded_model.eval()

```

Listing 2 – Sérialisation du state_dict

2.6. Évaluation et résultats

Métriques clés

- **Accuracy** : $\frac{VP+VN}{total}$
- **Precision / Recall / F1** via `classification_report`
- **Matrice de confusion** (visualisée avec Seaborn)

Le MLP atteint généralement une accuracy supérieure à **95%** sur ce jeu de données.

Limites du MLP sur données tabulaires

- Insensible à l'ordre des colonnes (pas de géométrie spatiale).
- Fortement sensible aux hyperparamètres (nombre d'époques, α).
- Risque de surapprentissage rapide sans régularisation.

3. Partie II – Réseau de Neurones Convolutif (CNN)

3.1. Idée centrale

Un CNN exploite la **structure spatiale** des images grâce au *partage de poids* : un même filtre est appliqué à toutes les positions de l'image. Cela réduit drastiquement le nombre de paramètres par rapport à un MLP qui aurait besoin d'un poids distinct pour chaque pixel.

3.2. Corrélation croisée 2D

L'opération de base du CNN (souvent appelée « convolution » par abus de langage) calcule, pour un noyau K de taille $h \times w$:

$$Y[i, j] = \sum_{a=0}^{h-1} \sum_{b=0}^{w-1} X[i+a, j+b] \cdot K[a, b]$$

La taille de sortie est $(I - h + 1) \times (J - w + 1)$ (sans padding, stride 1).

```
def corr2d(X, K):
    h, w = K.shape
    Y = torch.zeros((X.shape[0]-h+1, X.shape[1]-w+1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i+h, j:j+w] * K).sum()
    return Y
```

Listing 3 – Corrélacion croisée manuelle (PyTorch)

3.3. Max-Pooling

La couche de pooling réduit la résolution spatiale en conservant la valeur maximale dans une fenêtre de taille $p \times p$ (stride = p) :

$$Y[i, j] = \max_{a, b \in [0, p-1]} X[ip+a, jp+b]$$

```
def max_pooling2d(X, pool_size):
    h_out = X.shape[0] // pool_size
    w_out = X.shape[1] // pool_size
    Y = torch.zeros((h_out, w_out))
    for i in range(h_out):
        for j in range(w_out):
            Y[i, j] = X[i*pool_size:i*pool_size+pool_size,
                        j*pool_size:j*pool_size+pool_size].max()
    return Y
```

Listing 4 – Max-Pooling 2D manuel

3.4. Architecture LeNet-5 (CIFAR-10)

Couches de LeNet-5 adaptée à CIFAR-10 ($32 \times 32 \times 3$)

#	Couche	Paramètres	Sortie
1	Conv2d + ReLU	3 entrées $\rightarrow$ 6 filtres 5×5	$28 \times 28 \times 6$
2	MaxPool2d	fenêtre 2×2 , stride 2	$14 \times 14 \times 6$
3	Conv2d + ReLU	6 entrées $\rightarrow$ 16 filtres 5×5	$10 \times 10 \times 16$
4	MaxPool2d	fenêtre 2×2 , stride 2	$5 \times 5 \times 16$
5	Flatten	—	400
6	Linear + ReLU	400 $\rightarrow$ 120	120
7	Linear + ReLU	120 $\rightarrow$ 84	84
8	Linear	84 $\rightarrow$ 10	10 (logits)

```
class LeNet5(nn.Module):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, kernel_size=5)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.pool2 = nn.MaxPool2d(2, 2)
```

```

self.fc1 = nn.Linear(16*5*5, 120)
self.fc2 = nn.Linear(120, 84)
self.fc3 = nn.Linear(84, 10)

def forward(self, x):
    x = self.pool1(F.relu(self.conv1(x)))
    x = self.pool2(F.relu(self.conv2(x)))
    x = x.view(-1, 16*5*5)
    x = F.relu(self.fc1(x)); x = F.relu(self.fc2(x))
    return self.fc3(x)

```

Listing 5 – Définition de LeNet-5 en PyTorch

3.5. Préparation des données CIFAR-10

```

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
trainset = torchvision.datasets.CIFAR10(root='./data',
    train=True, download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=64, shuffle=True)

```

Listing 6 – Normalisation et DataLoader

3.6. Visualisation des feature maps

Après la première convolution, chacun des 6 filtres produit une *feature map* qui met en évidence un motif particulier de l'image (bords, textures, etc.). La visualisation de ces activations permet de comprendre ce que le réseau « voit ».

3.7. Comparaison CNN vs MLP

Critère	SimpleMLP	LeNet-5
Paramètres	~1.5 M	~61 k
Partage des poids	Non	Oui (filtres)
Invariance spatiale	Non	Partielle
Accuracy sur CIFAR-10	~55%	~65%

Conclusion CNN

Le CNN obtient une meilleure accuracy avec beaucoup moins de paramètres que le MLP. Un filtre qui détecte un bord vertical est utile partout dans l'image; le MLP, lui, doit réapprendre ce motif pour chaque position, ce qui mène au surapprentissage.

4. Partie III – Modèles Séquentiels (RNN / LSTM / GRU)

4.1. Modèles de langage

4.1.1. Objectif probabiliste

Un modèle de langage estime la probabilité d'une séquence de tokens $(x_1, \dots, x_T)$. Par la règle de chaîne :

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1})$$

La fonction de perte associée est la *log-vraisemblance négative* :

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_{<t})$$

4.1.2. Perplexité

Perplexité (PPL)

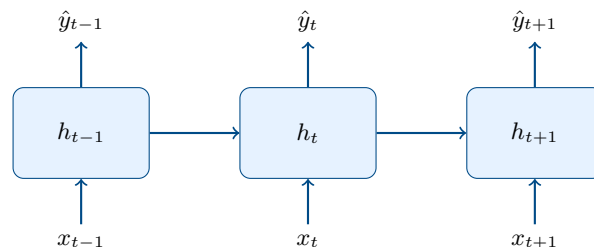
$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_{<t})\right)$$

Une PPL de K signifie que le modèle hésite entre K mots équiprobables à chaque étape.
Plus la PPL est faible, meilleur est le modèle.

4.2. Réseau neuronal récurrent (RNN)

4.2.1. Équation de récurrence

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h), \quad \hat{y}_t = \text{softmax}(W_{hq} h_t + b_q)$$



```
rnn = nn.RNN(input_size=32, hidden_size=64, batch_first=True)
X = torch.randn(16, 10, 32) # batch=16, longueur=10, dim=32
H0 = torch.zeros(1, 16, 64)
Y, Hn = rnn(X, H0)
# Y : (16, 10, 64)   Hn : (1, 16, 64)
```

Listing 7 – RNN minimal en PyTorch

4.3. Apprentissage : BPTT et gradient clipping

4.3.1. Rétropropagation à travers le temps (BPTT)

Pour entraîner un RNN on le *déplie* dans le temps, transformant le graphe récurrent en une chaîne de blocs identiques. Le gradient traversant toute la séquence contient un produit de jacobiniennes :

$$\frac{\partial \mathcal{L}}{\partial h_k} = \sum_{t \geq k} \frac{\partial \mathcal{L}}{\partial h_t} \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

Ce produit est à l'origine de deux pathologies classiques :

Phénomène	Cause	Remède
Gradient évanescent	Produit $\rightarrow 0$	LSTM / GRU
Gradient explosif	Produit $\rightarrow \infty$	Gradient clipping

4.3.2. Gradient clipping

$$g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) \cdot g$$

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step(); optimizer.zero_grad()
```

Listing 8 – Clipping dans PyTorch

Observation expérimentale : sans clipping, la norme L2 des gradients du SimpleRNN subit de violentes oscillations qui détruisent les représentations apprises. Avec clipping (`max_norm=1.0`), la norme reste stable.

4.4. LSTM : Long Short-Term Memory

4.4.1. Équations des portes

$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$	porte d'oubli
$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$	porte d'entrée
$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$	candidat
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	cellule de mémoire
$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$	porte de sortie
$h_t = o_t \odot \tanh(c_t)$	état caché

Rôle des portes LSTM

- f_t : quelle part de l'ancienne mémoire c_{t-1} garder.
- i_t : quelle information nouvelle écrire dans la cellule.
- $\tilde{c}_t$: contenu potentiel à stocker.
- o_t : quelle part de c_t exposer dans l'état caché.

Le chemin **additionnel** via c_t est à l'origine de la résistance du LSTM au gradient évanescent.

```
lstm = nn.LSTM(input_size=32, hidden_size=64, batch_first=True)
X = torch.randn(8, 15, 32)
Y, (Hn, Cn) = lstm(X)
# Y: (8, 15, 64)  Hn: (1, 8, 64)  Cn: (1, 8, 64)
```

Listing 9 – LSTM en PyTorch

4.5. GRU : Gated Recurrent Unit

$$\begin{aligned}
 z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{porte de mise à jour} \\
 r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{porte de réinitialisation} \\
 \tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) && \text{état candidat} \\
 h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t
 \end{aligned}$$

4.5.1. Comparaison LSTM / GRU

Critère	LSTM	GRU
Structure	3 portes + cellule	2 portes
Mémoire explicite	Oui, via c_t	Non, intégrée à h_t
Coût computationnel	Plus élevé	Plus léger
Usage typique	Longues dépendances	Bon compromis vitesse/précision

4.6. Variantes architecturales

4.6.1. RNN profonds

On empile plusieurs couches récurrentes :

$$h_t^{(\ell)} = f^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}), \quad \ell \geq 2$$

La couche basse capture des motifs locaux, la couche haute des représentations plus abstraites.

4.6.2. RNN bidirectionnels

$$\vec{h}_t = f(x_t, \vec{h}_{t-1}), \quad \overleftarrow{h}_t = g(x_t, \overleftarrow{h}_{t+1}), \quad h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Limite

Les RNN bidirectionnels sont adaptés à l'étiquetage ou la reconnaissance d'entités nommées, mais **impossibles** en génération auto-régressive (on ne connaît pas le futur pendant l'inférence).

4.7. Analyse comparative des modèles de langage

Les trois architectures (SimpleRNN, LSTM, GRU) sont entraînées sur le corpus anglais issu de la traduction fra-eng (jeu de données Tatoeba) :

Modèle	PPL train	PPL val	Temps/époque	Paramètres
SimpleRNN	élevée	élevée	rapide	moyen
LSTM	faible	faible	lent	élevé
GRU	faible	faible	moyen	réduit

Observation : le SimpleRNN souffre d'une baisse rapide de performance en raison de l'atténuation du gradient. Le LSTM mémorise mieux le contexte lointain. Le GRU offre un excellent compromis performance / vitesse.

5. Partie IV – Séquence-à-séquence (Seq2Seq) et Traduction

5.1. Préparation des données

5.1.1. Pipeline

1. Nettoyage du texte (unicode → ASCII, minuscules, ponctuations).
2. Tokenisation par espaces.
3. Construction du vocabulaire (top-5000 tokens).
4. Ajout de tokens spéciaux.
5. Padding dynamique par mini-lot (`pad_sequence`).
6. Séparation 80/10/10 (train/val/test).

5.1.2. Tokens spéciaux

Indice	Token	Rôle
0	<PAD>	Remplissage
1	<SOS>	Début de séquence
2	<EOS>	Fin de séquence
3	<UNK>	Mot inconnu

5.1.3. Perte masquée

Pour ignorer les positions de padding dans le calcul de la perte :

$$\mathcal{L} = - \frac{\sum_t m_t \log P(y_t^* | y_{<t}, x)}{\sum_t m_t}$$

où $m_t = 1$ pour une position valide et $m_t = 0$ pour un padding.

5.2. Architecture encodeur-décodeur

Formalisation

On modélise :

$$P(y_{1:T_y} | x_{1:T_x}) = \prod_{t=1}^{T_y} P(y_t | y_{<t}, x_{1:T_x})$$

L'encodeur produit un contexte c (ou des états cachés $h_1, \dots, h_{T_x}$). Le décodeur conditionne sa génération sur ce contexte.

5.3. Seq2Seq récurrent

5.3.1. Encodeur LSTM bidirectionnel

```
class EncoderSeq2Seq(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                             batch_first=True, bidirectional=True)
        self.fc_h = nn.Linear(hidden_dim*2, hidden_dim)
        self.fc_c = nn.Linear(hidden_dim*2, hidden_dim)
```

```
def forward(self, src):
    embedded = self.embedding(src)
    outputs, (hidden, cell) = self.lstm(embedded)
    # Concaténation des directions
    h = torch.tanh(self.fc_h(
        torch.cat([hidden[0::2], hidden[1::2]], dim=-1)))
    c = torch.tanh(self.fc_c(
        torch.cat([cell[0::2], cell[1::2]], dim=-1)))
    return outputs, (h, c)
```

Listing 10 – Encodeur bidirectionnel

5.3.2. Teacher Forcing

Teacher Forcing

Pendant l'entraînement, au lieu d'injecter la prédiction $\hat{y}_{t-1}$ dans le décodeur, on lui fournit le vrai token y_{t-1}^* .

- **Avantage** : convergence plus rapide.
- **Limite (Exposure Bias)** : décalage entre train et inférence. Un ratio hybride ($p \approx 0.5$) équilibre les deux dynamiques.

5.3.3. Inférence auto-régressive

```
for t in range(max_len):
    prediction, decoder_hidden = decoder(dec_input, decoder_hidden)
    pred = prediction.argmax(dim=-1)
    if pred.item() == EOS_IDX:
        break
    decoded_words.append(tgt_vocab.index2word[pred.item()])
    dec_input = pred.unsqueeze(1)
```

Listing 11 – Génération token par token

5.4. Stratégies de décodage

5.4.1. Décodage glouton (*Greedy*)

À chaque pas, on choisit le token le plus probable :

$$y_t = \arg \max_y P(y \mid y_{<t}, x)$$

Rapide mais **myope** : un mauvais choix initial fausse toute la suite.

5.4.2. Score d'une séquence

$$\log P(y_{1:T} \mid x) = \sum_{t=1}^T \log P(y_t \mid y_{<t}, x)$$

On travaille en log-probabilités pour éviter les sous-flux numériques et transformer le produit en somme.

5.4.3. Beam Search

Algorithme Beam Search (largeur B)

1. Initialiser le faisceau avec $\langle \text{SOS} \rangle$.
2. Développer chaque hypothèse par tous les tokens possibles.
3. Recalculer les scores.
4. Conserver les B meilleures hypothèses partielles.
5. Répéter jusqu'à $\langle \text{EOS} \rangle$ ou longueur maximale.

5.4.4. Pénalisation de longueur

Sans correction, le beam search favorise les séquences courtes. On normalise :

$$\text{score}(y) = \frac{1}{T^\alpha} \sum_{t=1}^T \log P(y_t \mid y_{<t}, x), \quad \alpha \in [0, 1]$$

5.5. Évaluation : score BLEU

Le score BLEU mesure la qualité de la traduction via la précision des n -grammes, pondérée par une pénalité de brièveté.

Mécanisme	BLEU-1	BLEU-4
Greedy Decoding	moderate	faible
Beam Search ($B = 5$)	plus élevé	plus élevé

Sensibilité au beam size

Le score BLEU-4 croît avec B mais se stabilise (voire décroît) pour de grands B à cause de l'accumulation de log-probabilités négatives sur de longues séquences. Un $B \in [3, 8]$ est souvent un bon compromis.

6. Comparaison globale des architectures

Modèle	Atouts	Limites
MLP	Polyvalent, rapide, puissant sur données tabulaires	Pas de géométrie spatiale ni temporelle
CNN	Partage de poids, invariance spatiale, efficace sur images	Fenêtre receptive fixe, pas de mémoire temporelle
RNN simple	Léger, conceptuellement simple	Gradient évanescent / explosif, dépendances courtes
LSTM	Excellent contrôle de la mémoire, robuste	Coûteux, nombreux paramètres
GRU	Plus simple que LSTM, souvent aussi performant	Légèrement moins expressif
RNN profond	Représentations hiérarchiques	Entraînement délicat, coût supérieur
Bi-RNN	Utilise passé et futur simultanément	Inadapté à la génération causale
Seq2Seq	Modélise proprement source et cible	Compression du contexte parfois insuffisante
Beam search	Meilleure exploration que greedy	Plus lent, sensible au réglage de B et α

7. Mini-résumé opérationnel

1. Un **MLP** apprend des représentations statistiques à partir de données tabulaires normalisées ; l'initialisation Xavier et Adam suffisent dans la plupart des cas.
2. Un **CNN** exploite la structure spatiale des images grâce au partage de poids ; il surpasse le MLP avec un ordre de grandeur moins de paramètres.
3. Un **RNN** introduit une mémoire cachée transmise de pas en pas pour traiter les séquences.
4. La **BPTT** entraîne le RNN, mais souffre de gradients évanescents ou explosifs ; le clipping pallie l'explosion.
5. Les **LSTM** et **GRU** stabilisent l'apprentissage grâce à des portes de contrôle de flux.
6. Les versions **profondes** et **bidirectionnelles** enrichissent les représentations au prix d'un coût supérieur.
7. Pour la traduction, un pipeline propre (tokenisation, vocabulaire, padding, masquage) est indispensable.
8. L'architecture **encodeur-décodeur** sépare la compréhension de la génération et modélise la traduction conditionnelle.
9. Le **Seq2Seq** s'entraîne avec teacher forcing et s'évalue par le score BLEU.
10. Le **beam search** améliore l'inférence en explorant plusieurs hypothèses simultanément.

8. Squelette d'entraînement Seq2Seq complet

```
encoder = EncoderSeq2Seq(src_vocab.num_words, 128, 256, 2).to(DEVICE)
decoder = DecoderSeq2Seq(tgt_vocab.num_words, 128, 256, 2).to(DEVICE)
```

```
seq2seq = Seq2Seq(encoder, decoder).to(DEVICE)

optimizer = optim.Adam(seq2seq.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)

for epoch in range(10):
    seq2seq.train()
    epoch_loss = 0
    for src, tgt in train_loader:
        src, tgt = src.to(DEVICE), tgt.to(DEVICE)
        optimizer.zero_grad()
        output = seq2seq(src, tgt, teacher_forcing_ratio=0.5)
        loss = criterion(
            output[:, 1:, :].reshape(-1, output.shape[-1]),
            tgt[:, 1:].reshape(-1))
        loss.backward()
        torch.nn.utils.clip_grad_norm_(seq2seq.parameters(), 5.0)
        optimizer.step()
        epoch_loss += loss.item()
    print(f"Epoch {epoch+1} | Loss: {epoch_loss/len(train_loader):.4f}")
```

Listing 12 – Pipeline Seq2Seq – entraînement et inférence

9. Conclusion

Ce document met en évidence une continuité pédagogique forte à travers les trois notebooks. On part d'une classification tabulaire avec un MLP, on introduit l'invariance spatiale avec un CNN, puis on modélise le temps avec les architectures récurrentes. La chaîne complète est :

**MLP → CNN → RNN → LSTM/GRU → Encodeur-Décodeur →
Seq2Seq → Beam Search**

Cette chaîne constitue une base solide pour comprendre les architectures classiques de Deep Learning avant de passer à des approches plus modernes comme l'attention (Bahdanau) et les Transformers.